LKP Project: Enhancing OuicheFS with Sliced Block Storage

Authors: Jonathan Lyssy and Niels Freitag
Date: August 2025

Introduction

OuicheFS is a custom file system with a focus on simplicity and performance. In this project, we extended OuicheFS by adding support for sliced block storage, aiming to improve efficiency for small file storage.

This report outlines the design decisions behind our implementation, a status report on each feature in the development roadmap (tasks 1.2 to 1.11), and experimental results comparing the modified version with the baseline OuicheFS.

Functional Features

Patch: Implementation of Read and Write without Page Cache

This patch implements reads and writes without using the page cache. The resulting functions are rather similar to the original versions, with the main difference being direct writes with `sb_bread` and `brelse` instead of the page cache.

Patch: Sliced Block Storage

This patch implements the requested features of 1.3 to 1.4. A new block struct `ouichefs_sliced_block` is introduced which has 32 slices with the first used as a header to store metadata and the rest for data (1.3).

For checks the state is accessible in a virtual file system (1.4) under `/sys/fs/ouichefs/<partition>/`. Most of the stats were already stored in the superblock info and could be accessed via the sysfs show. To ensure that values such as the number of free blocks are accurate, the superblock is updated with each allocation and put. Used blocks are counted by calculating the number of blocks, excluding those for the superblock and metadata. Free slice counting is done by iterating over all sliced blocks and using their bitmap to count the free slices. To make access easier in certain areas, we added a superblock reference to the superblock info struct.

Patch: Write for Slices

This patch implements the features 1.5 to 1.9
In order to handle small files with our sliced block storage, we assumed that each small file would fit into a single slice. This patch implements writing to small slices (1.5).

To help debugging we added an ioctl command as a read operation to display the contents of sliced blocks. As the header is also given here we decided to have it printed as hexadecimal value while the rest is printed as characters. While this might be bad for file formats that have non ASCII characters for testing this for most purposes seemed to be the better choice. (This could easily be changes however.)

The deletion of files within a single slice has been added. When deleting small files, the header of the sliced block is updated to reflect any changes in status (e.g. from full to partially filled or from partially filled to empty) and the partially filled list is updated accordingly. If the last slice is deleted, the entire block is released; otherwise, only the slice is freed.
To enable the consistent storage of the files between reboots and update the sysfs stats we use, the the updated free slice counts are written to the superblock on disk.

To enable sliced blocks and legacy storage (1.8), we added conditional implementations of read and write that check the data size and select the appropriate storage method. Notably, if a small file has more data written to it than can fit into a slice, the slice is removed and the data is moved to legacy storage. However, transitioning back is not part of this patch. Legacy storage still requires at least two blocks for storage, as the full block is used for data, whereas sliced storage uses the first slice for metadata.

Reading for small files is added (1.9) which is done by finding the corresponding slice of the file and reading the requested size.

Patch: Multi-Slice Files

For files that span multiple slices, our file system uses a specific strategy for allocating and growing blocks and slices.
The general idea is that, if there are files in a sliced block that already span multiple slices if the number of required block does not change the write is simply done with the first block as the start position.

If the block number changes the first-fit algorithm is used on the partially filled list, taking advantage of a largest-gap parameter to find a suitable slice. The largest gap parameter is stored for each file in its first slice as metadata and updated on each write or remove.
Once a sliced block with enough space is found, best fit is used to determine where in the block the file should be saved. Previous data slices of that file are stored to make space for the new starting slice, and then copied to the new position before the user buffer is written. Shrinking simply involves first writing the data and then freeing all unnecessary slices.

Implemented but Not Fully Functional Features

Multi-slice file operations are functional but not optimal. Currently, any change in slice count triggers a full relocation of the file. While this reduces fragmentation, it introduces overhead even for small size changes.

Limitations: - No in-place extension/shrink when adjacent slices are free. - Inefficient for workloads with frequent small writes.

Potential Fixes:
Implement smarter allocation heuristics that check for available adjacent slices before relocating when growing and when shrinking just free slices that are not used anymore.

Not Implemented Features

A feature that would be quite helpful would be a defragmentation algorithm to efficiently reorder the storage in sliced blocks whenever the amount of partially filled blocks becomes rather big. A few concept we suggest would be to focus on blocks that are filled less than 50% as these require less data moving. These can then be sorted in ascending order of size and start reordering to the other partially filled blocks. Further improvements we suggest would be to also prioritize blocks with less different files to reduce inode updates. A more advanced optimization would be to some degree considering the time since last write to position files that are unlikely to change is size at positions where they fit well as long as they do not grow or shrink.

The second feature of version 1.11 is that legacy file blocks are shrunk into slices when a legacy file is small enough to fit into a sliced block. When the write is complete, the size change is detected and the remaining data from legacy storage is combined with the write and stored in a slice. The position can be found in the same way as for any sliced file. Conversion from sliced to legacy also works, which is more important for functionality.

Conclusion

Experimental results

fio Throughput Comparison We benchmarked random 4KiB read/write using fio on both the v.6.5.7 and our new OuicheFS implementations.

Old implementation:
- Read: 23.8 MiB/s, 6092 IOPS, avg latency 163 us
- Write: 23.8 MiB/s, 6102 IOPS, avg latency 39 us

New implementation:
- Read: 31.4 MiB/s, 8028 IOPS, avg latency 10 us
- Write: 31.4 MiB/s, 8031 IOPS, avg latency 155 us

Summary:
The new version improves read/write throughput by ~30% and increases IOPS by a similar margin. Average read latency dropped significantly, while write latency increased, likely due to more advanced allocation logic. Overall, the new allocation strategies yield much better random IO performance, especially for reads.

OuicheFS Storage Efficiency Comparison New implementation (with slices):
- 100 small files (random sizes, total 210,698 bytes data)
- Disk used: 294,912 bytes
- **Efficiency:** 71% (from sysfs)

Old implementation (no slices, no sysfs):
- 100 small files would each use 2 blocks (8192 bytes), totaling **819,200 bytes** used for the same data
- **Efficiency:** 26% (210,698 / 819,200 × 100)

Conclusion:
The new implementation greatly improves storage efficiency for small files, increasing it from ~26% to 71%.

Summary and Outlook

Our project changes the OuicheFS implementation of a filesystem to also support sliced blocks to improve storage efficiency. Regarding implementation decisions we tested a couple different approaches but often the results showed the algorithms to be best based on the situation in most cases.
For the multislice block allocation we ended up using first fit for the block and best fit for the slices with only the largest gap being stored. Our tests here showed no significant improvements of using anything more than first fit for the blocks while for the slices first fit worked quite well we stayed with best fit as it performed a little better in regard to fragmentation while testing as long as files sizes do not change too often. Despite having a lot of space in the first slice for more metadata we decided to only store the size of the largest gap to make the first fit run easier. Storing more info could improve the allocation strategy but the frequent requirement to update a lot of data on every write or remove made us decide against using it.
In general, this file system performs significantly better when used with smaller files, particularly when applied to contiguous slices for the same files. It has been demonstrated to be particularly effective for average file sizes.